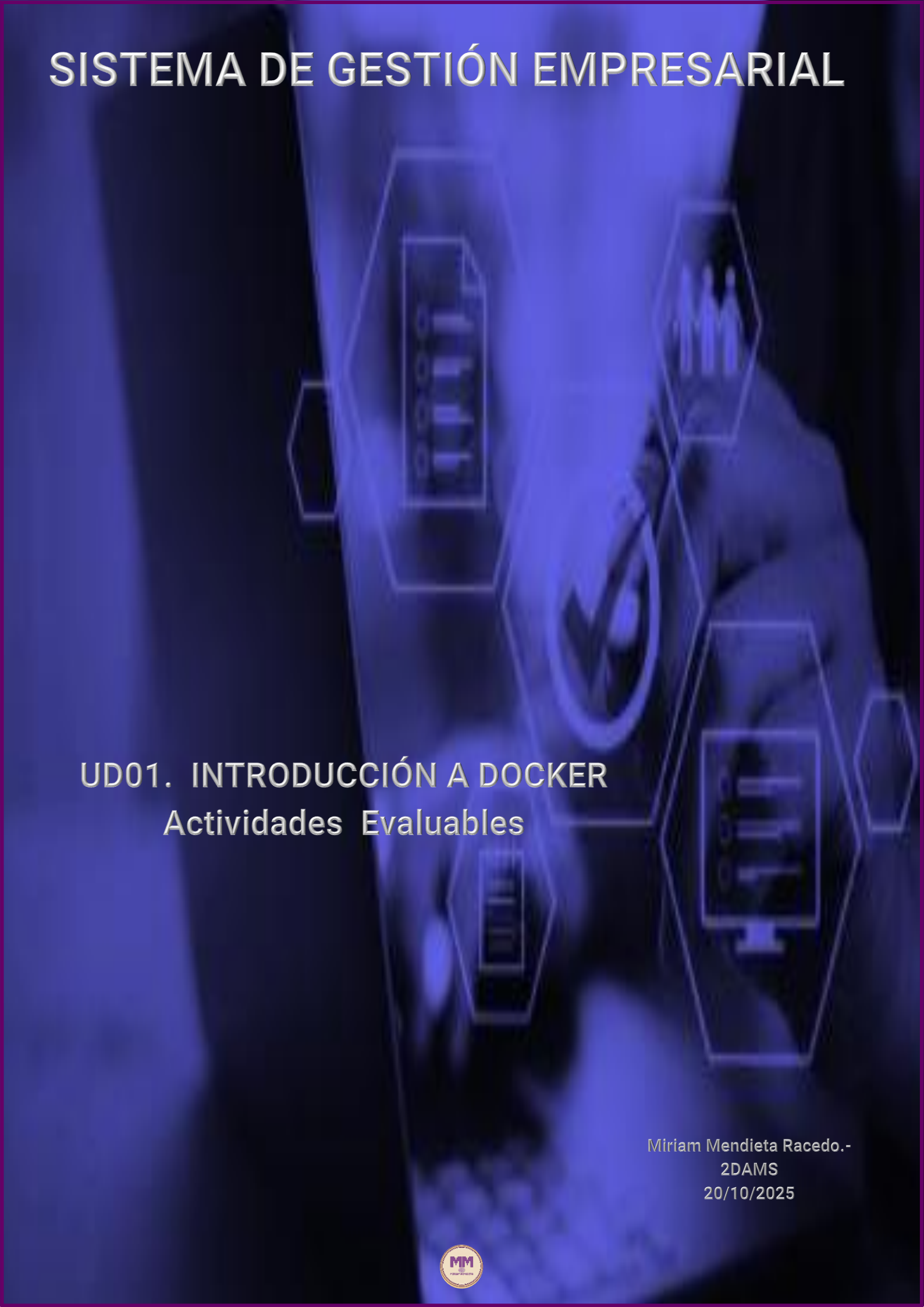
SISTEMA DE GESTIÓN EMPRESARIAL



UD01. INTRODUCCIÓN A DOCKER
Actividades Evaluables

Miriam Mendieta Racedo.-
2DAMS
20/10/2025

ÍNDICE

INTRODUCCIÓN **1**

ACTIVIDAD 01 2
ACTIVIDAD 02 11
ACTIVIDAD 03 14
ACTIVIDAD 04 19
ACTIVIDAD 05 22

BIBLIOGRAFIA **23**

INTRODUCCIÓN

En el mundo del desarrollo de software, existe un desafío constante y frustrante: la inconsistencia del entorno. La típica frase " en mi máquina funciona " ha atormentado a todo equipos de desarrollo que se precie… donde una aplicación perfecta se desmorona al moverse del ordenador del desarrollador al servidor de pruebas o a producción.

En este contexto la necesidad de entornos consistentes, portables y fácilmente replicables ha impulsado el uso de tecnologías como **Docker**. Esta herramienta ofrece un enfoque simple pero poderoso para empaquetar una aplicación y todas sus dependencias —bibliotecas, archivos de configuración e incluso el entorno operativo— en una unidad estandarizada llamada Imagen. Esta Imagen actúa como un plano inmutable y el **Contenedor**, instancia viva, ligera y aislada, que garantiza que el entorno de ejecución sea idéntico en cualquier lugar.

A lo largo de esta serie de ejercicios, se explora los fundamentos de Docker y su aplicación práctica en distintos tipos de escenarios.

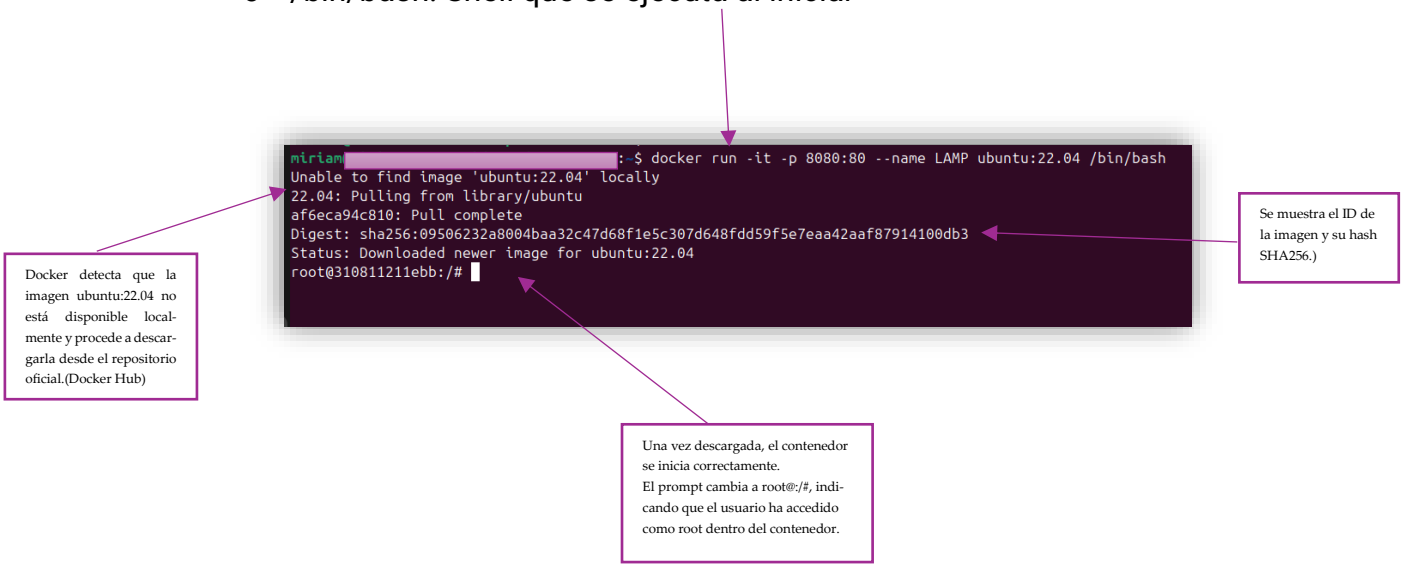
ACTIVIDAD 01

Caso práctico 02 - Instalando LAMP + Wordpress en contenedor

1. Inicio de contenedor Docker para entorno LAMP

Se trabaja desde una terminal Ubuntu, donde se ejecuta el comando para iniciar un contenedor Docker con Ubuntu 22.04, destinado a configurar un entorno LAMP (Linux, Apache, MySQL/MariaDB, PHP).

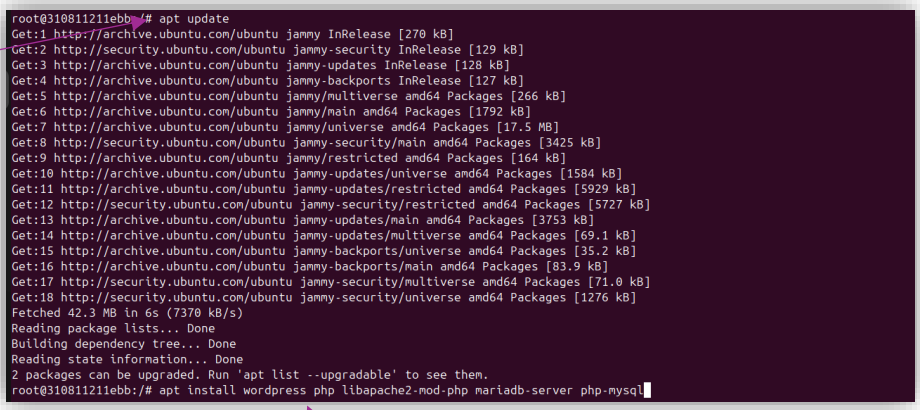
- -it: Modo interactivo con terminal
- -p 8080:80: Redirige el puerto 8080 del host al 80 del contenedor
- --name LAMP: Asigna el nombre "LAMP" al contenedor
- ubuntu:22.04: Imagen base utilizada
- /bin/bash: Shell que se ejecuta al iniciar



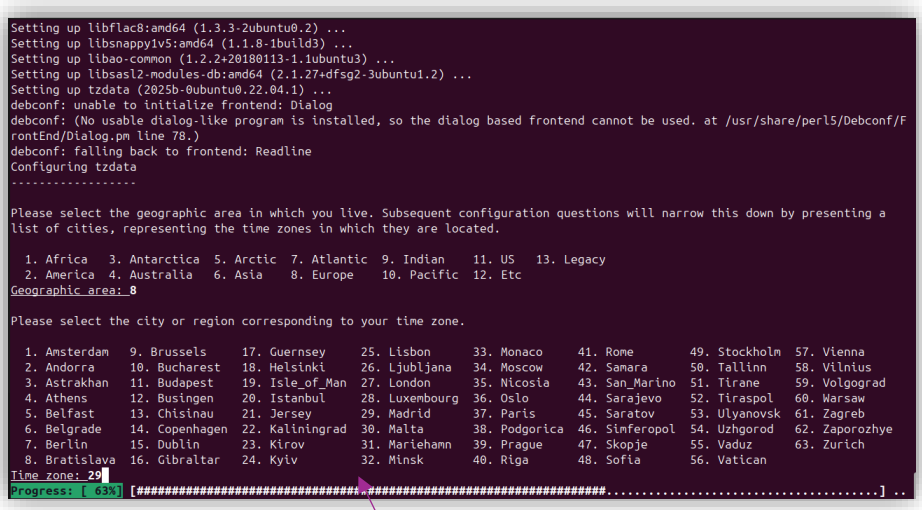
2. **Instalando LAMP Y Wordpress**

Se preparar el entorno de ejecución de WordPress dentro del contenedor para garantizar que Apache pueda servir contenido dinámico y se establece la conexión entre WordPress y la base de datos MariaDB.

Actualización de repositorios



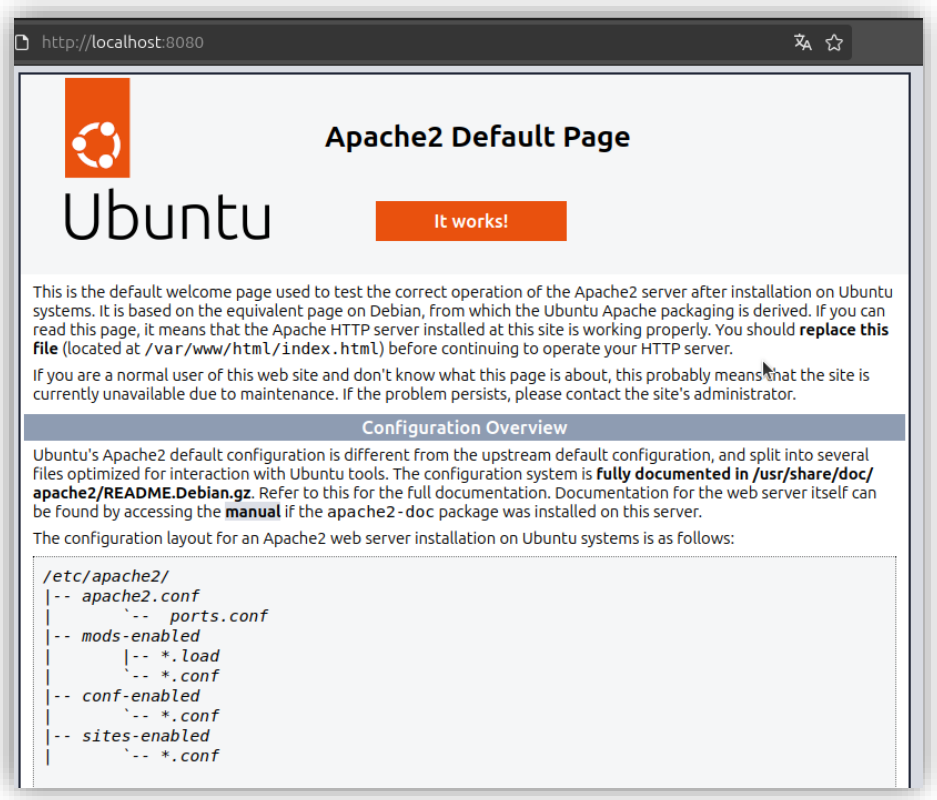
- apt install: gestor de paquete que permite la instalación.
- wordpress: CMS que se instalará en /usr/share/wordpress.
- php: Lenguaje de programación para el backend.
- libapache2-mod-php: Módulo que permite a Apache interpretar PHP.
- mariadb-server: Motor de base de datos compatible con MySQL.
- php-mysql: Extensión que permite a PHP interactuar con MariaDB.



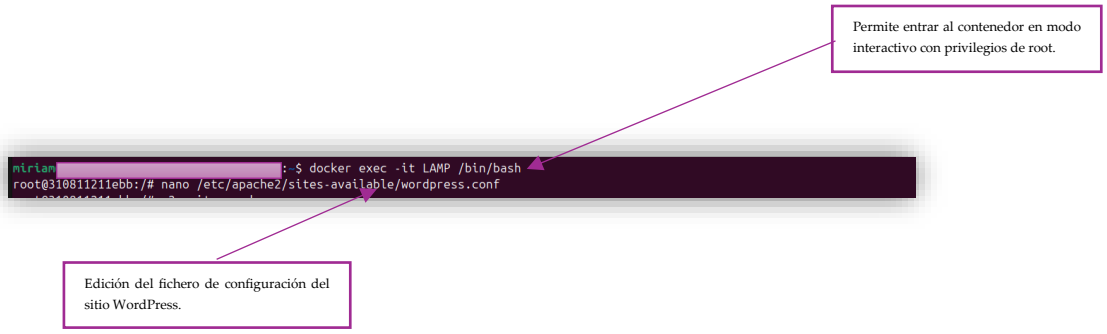
El sistema solicita al usuario seleccionar una zona geográfica. En este caso, se elige la opción 8, correspondiente a Europa.
Luego se presenta una lista de ciudades europeas. El usuario selecciona la opción 29 (Madrid).
Asegura que los registros de Apache, MariaDB y WordPress tengan la hora correcta.

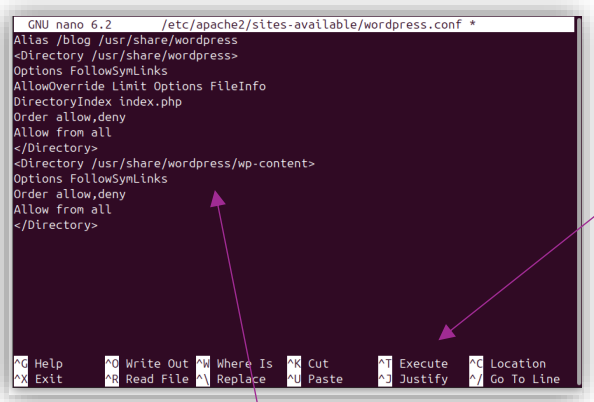
- Se lanza el servicio Apache con el comando: ***service apache2 start***

  Y se comprueba : ***http://localhost:8080***



- Preparando Apache para usar Wordpress

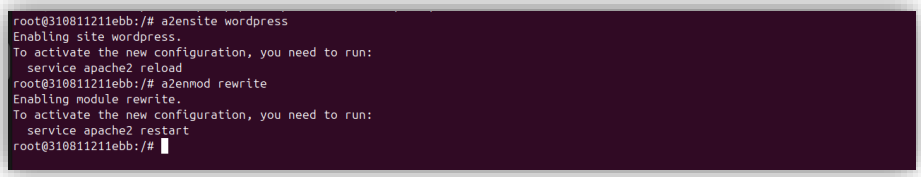




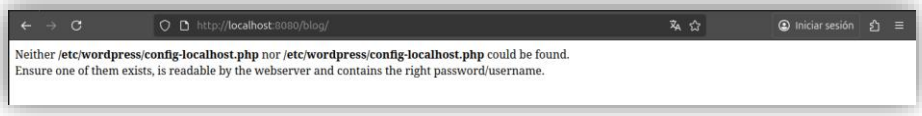
Guarda y cierra Nano
Ctrl + O → Guardar
Enter → Confirmar nombre
Ctrl + X → Salir

Alias /blog /usr/share/wordpress	Permite acceder a WordPress desde http://localhost:8080/blog
<Directory /usr/share/wordpress>	Define permisos y comportamiento del directorio principal
Options FollowSymLinks	Permite seguir enlaces simbólicos (útil para plugins y temas)
AllowOverride Limit Options FileInfo	Permite que .htaccess controle ciertas directivas
DirectoryIndex index.php	Define el archivo principal que se carga por defecto
Order allow,deny + Allow from all	Permite acceso público al directorio
<Directory /usr/share/wordpress/wp-content>	Asegura que los recursos como imágenes y temas sean accesibles

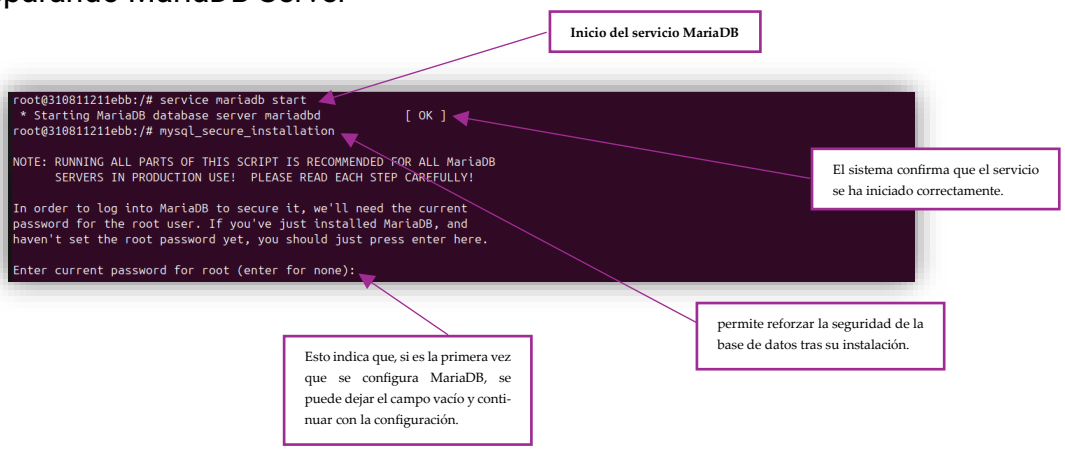
Cargar el sitio, habilitar “URL rewriting” y recargar Apache:

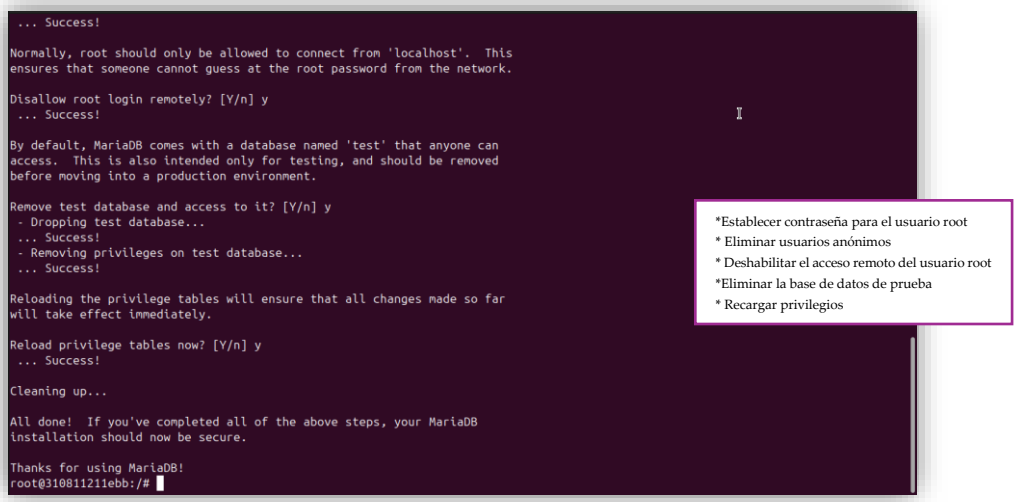


Comprobación con la URL http://localhost:8080/blog, aunque se ve un error porque aún no hemos configurado el fichero “/etc/wordpress/config-localhost.php”

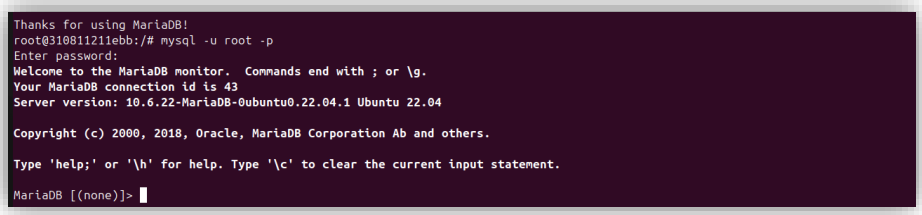


❖ Preparando MariaDB Server

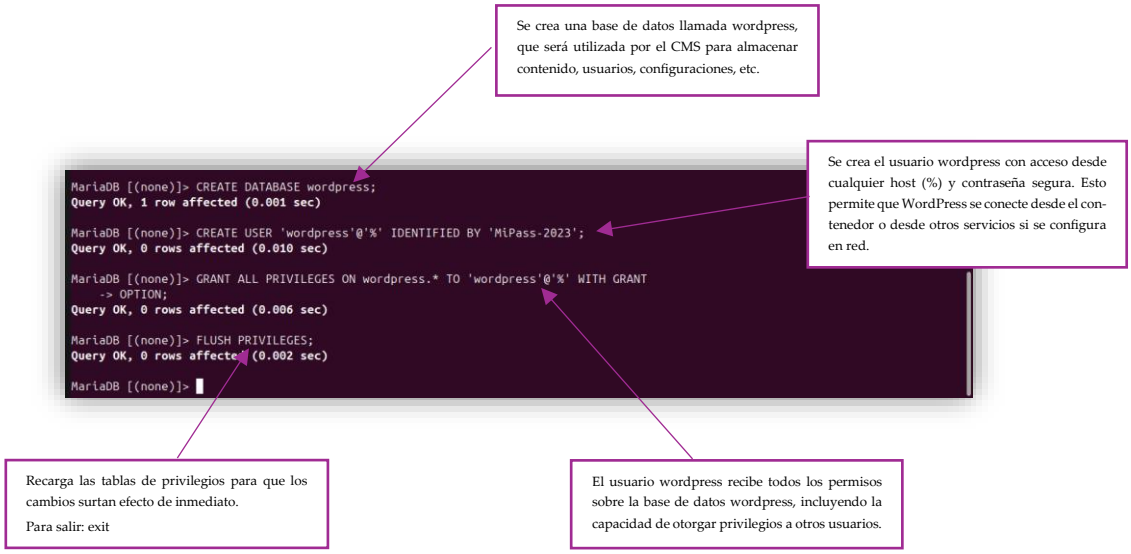




Se accede a la base de datos con el cliente MySQL de la siguiente forma:

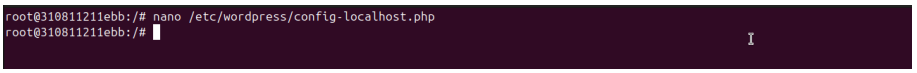


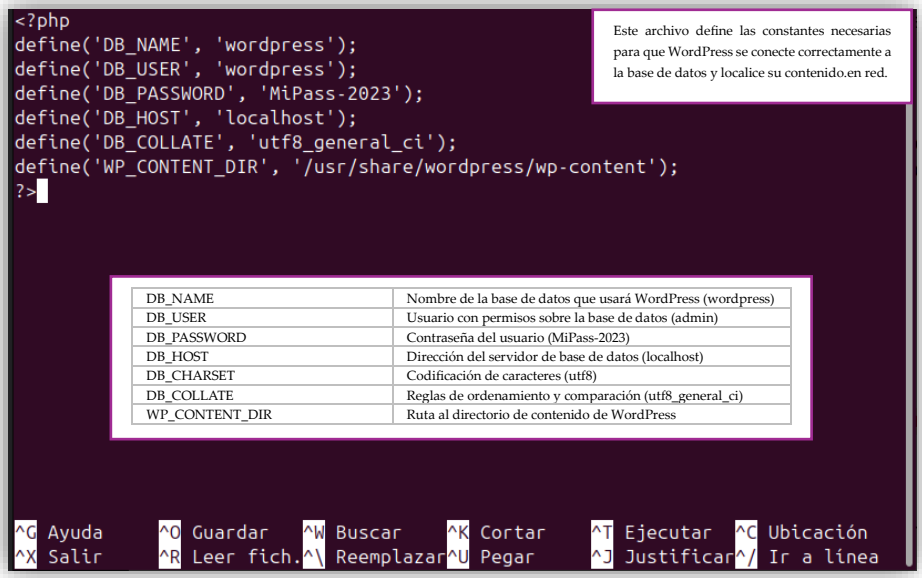
Creación de base de datos y usuario para WordPress en MariaDB



❖ Configurando Wordpress

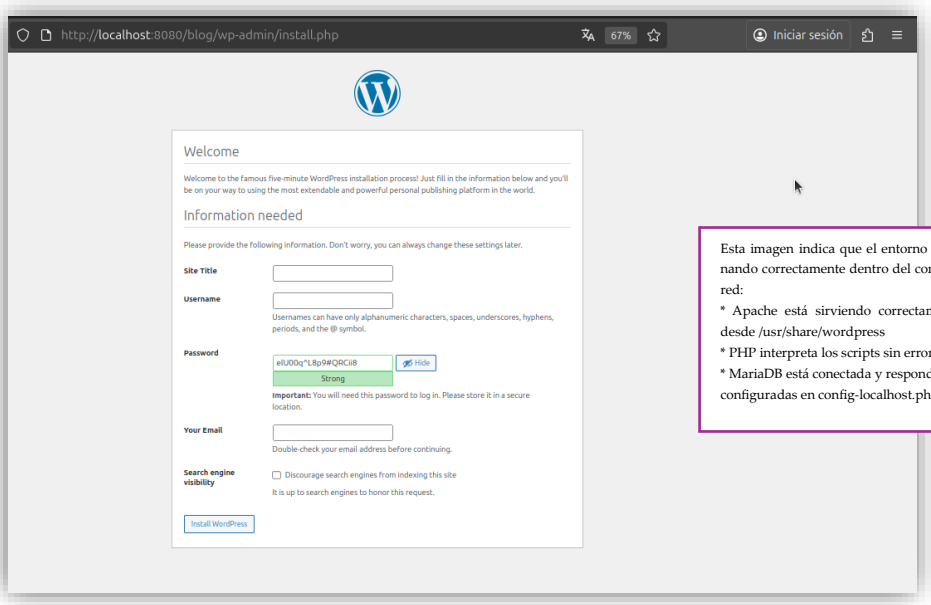
Crear (si no existe) o editar el fichero de configuración de Wordpress:





DB_NAME	Nombre de la base de datos que usará WordPress (wordpress)
DB_USER	Usuario con permisos sobre la base de datos (admin)
DB_PASSWORD	Contraseña del usuario (MiPass-2023)
DB_HOST	Dirección del servidor de base de datos (localhost)
DB_CHARSET	Codificación de caracteres (utf8)
DB_COLLATE	Reglas de ordenamiento y comparación (utf8_general_ci)
WP_CONTENT_DIR	Ruta al directorio de contenido de WordPress

Se accede a la URL **http://localhost:8080/blog**, donde se puede configurar el sitio Wordpress, indicando nombre de sitio, usuario y email.



Esta imagen indica que el entorno LAMP está funcionando correctamente dentro del contenedor Docker en red:
* Apache está sirviendo correctamente el contenido desde /usr/share/wordpress
* PHP interpreta los scripts sin errores
* MariaDB está conectada y responde a las credenciales configuradas en config-localhost.php

❖ Configurando los servicios para que arranquen al iniciar el contenedor

```
miriam:~$ docker exec -it LAMP /bin/bash
root@310811211ebb:/# cd /root
root@310811211ebb:~# nano .bashrc
root@310811211ebb:~# docker stop LAMP
```

Permite añadir comandos que se ejecutarán automáticamente al iniciar bash

Parar el contenedor

```
 GNU nano 6.2                                  .bashrc *
    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'
fi

# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
#if [ -f /etc/bash_completion ] && ! shopt -oq posix; then
#    . /etc/bash_completion
#fi
service apache2 start
service mariadb start

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute   ^C Location  M-U Undo    M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo    M-6 Copy
```

Se debe ir al final del archivo y colocar estos 2 comandos que permiten el arranque de los servicios desde el inicio.

```
miriam:~$ docker start LAMP
LAMP
miriam:~$
```

Inicia un contenedor previamente creado y detenido.

La terminal responde con LAMP, confirmando que el contenedor se ha iniciado correctamente.

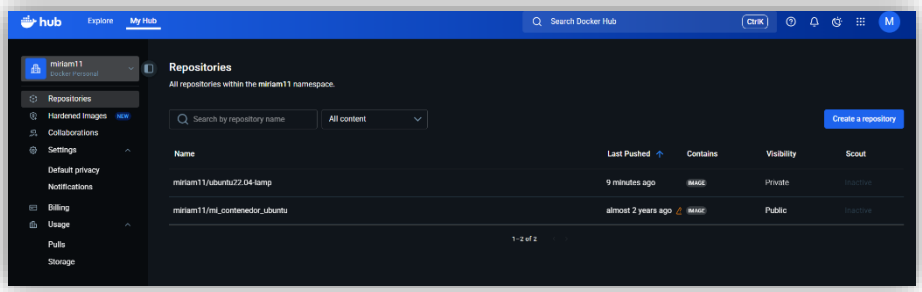
❖ Subiendo el contenedor a Docker Hub

```
miriam                          :~$ docker tag ubuntu:22.04 miriam11/ubuntu22.04-lamp
miriam                          :~$ docker info
Client: Docker Engine - Community
 Version:    28.4.0
 Context:    default
 Debug Mode: false
 Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.28.0
    Path:     /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version:  v2.39.4
    Path:     /usr/libexec/docker/cli-plugins/docker-compose

Server:
 Containers: 3
  Running: 0
  Paused: 0
  Stopped: 3
 Images: 2
 Server Version: 28.4.0
 Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Using metacopy: false
  Native Overlay Diff: true
  userxattr: false
 Logging Driver: json-file
 Cgroup Driver: systemd
 Cgroup Version: 2
 Plugins:
```

```
 containerd version: 05044ec0a9a75232cad458027ca83437aae3f4da
 runc version: v1.2.5-0-g59923ef
 init version: de40ad0
 Security Options:
  apparmor
  seccomp
   Profile: builtin
  cgroupns
 Kernel Version: 6.14.0-29-generic
 Operating System: Ubuntu 24.04.3 LTS
 OSType: linux
 Architecture: x86_64
 CPUs: 4
 Total Memory: 7.544GiB
 Name: miriam-Lenovo-ideapad-320-15ISK
 ID: 44955c65-5849-47c5-ab1a-b4b07ff30909
 Docker Root Dir: /var/lib/docker
 Debug Mode: false
 Username: miriam11
 Experimental: false
 Insecure Registries:
  ::1/128
  127.0.0.0/8
 Live Restore Enabled: false

miriam                          :~$ docker push miriam11/ubuntu22.04-lamp
Using default tag: latest
The push refers to repository [docker.io/miriam11/ubuntu22.04-lamp]
767e56ba346a: Mounted from library/ubuntu
latest: digest: sha256:89eecb4e2b529c4550b4fb475c2de8c3a19e63697265714aaffe9f443c221187 size: 529
```



UD 03. Caso práctico 03 - Accediendo a interfaz gráfica con NoVNC

1. Preparación del contenedor

```
miriam                                    :~$ docker run --rm -it -p 8080:8080 theasp/novnc
Unable to find image 'theasp/novnc:latest' locally
latest: Pulling from theasp/novnc
10ffc47270cd: Pull complete
2b363779fc2a: Pull complete
dc199bc6b323: Pull complete
Digest: sha256:244a8e4fd7b49532b1b264764e11af2f6ea2801c725bf09adcdaed155ca1cfa0
Status: Downloaded newer image for theasp/novnc:latest
+ RUN_FLUXBOX=yes
+ RUN_XTERM=yes
+ case $RUN_FLUXBOX in
+ case $RUN_XTERM in
+ exec supervisord -c /app/supervisord.conf
```

- --rm → Elimina el contenedor al cerrarlo (ideal para pruebas)
- -it → Modo interactivo con terminal
- -p 8080:8080 → Expone el puerto 8080 del contenedor al host (para acceder vía navegador)
- theasp/novnc → Imagen que ya incluye servidor VNC + cliente NoVNC listo para usar

2. Acceder a la interfaz gráfica

- Una vez que el contenedor esté corriendo:
- Abrir el navegador.
- Ir a http://localhost:8080/vnc.html
- Se cargará el cliente **NoVNC** (HTML5 + JavaScript).
- Hacer clic en **Connect** para iniciar la sesión VNC.
- Interfaz gráfica dentro del contenedor.





La imagen theasp/novnc suele incluir un entorno gráfico ligero como XFCE o similar, ideal para pruebas. Se puede interactuar con él como si fuera un escritorio remoto.

Al cerrar la terminal, el contenedor se elimina (por el parámetro --rm), si se quiere mantenerlo activo, elimina --rm y añade -d para modo daemon:

docker run -d -p 8080:8080 theasp/novnc

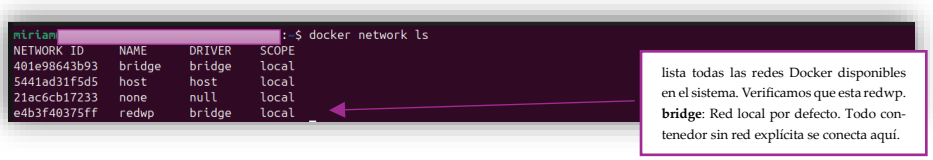
ACTIVIDAD 02

UD 05. Caso práctico 01- Wordpress + MariaDB

Se debe poner en marcha el popular CMS Wordpress. Para ello usaremos una red donde se conectarán dos contenedores: el primero utilizando Apache + PHP, junto con una versión instalada de Wordpress, mientras que el segundo contendrá un servidor de bases de datos MariaDB. Además, realizaremos un ejemplo de una migración de versión del contenedor MariaDB.

1. CREACIÓN DE LA RED





2. CREACIÓN CONTENEDOR MARIADB



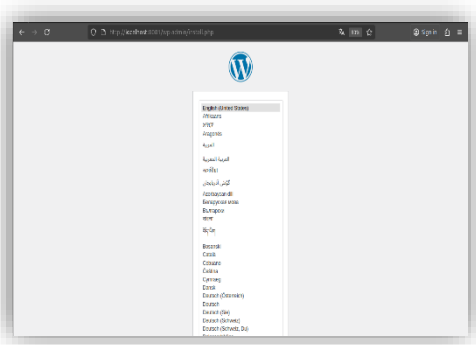
3. CREACIÓN CONTENEDOR CON WORDPRESS

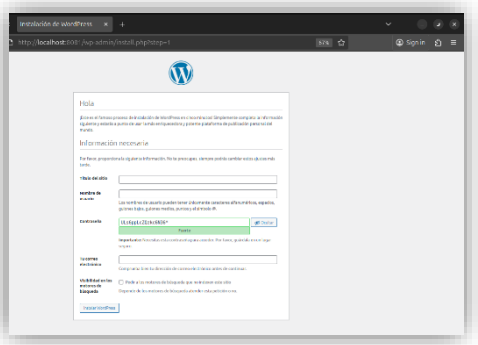
```
Deleted: sha256:6812608634d4e2652343b055d64b39334058d022a754469b1525fcee7c185601
Deleted: sha256:1d46119d249f7719e1820e24a311aa7c453f166f714969cffe89504678eaa447
miriam                                  $ docker run --name nuestrowp --network redwp -p 8080:80 -e WORDPRESS_DB_HOST=nuestromar
iadb -e WORDPRESS_DB_USER=cefireuser -e WORDPRESS_DB_PASSWORD=cefirepass -e WORDPRESS_DB_NAME=cefiredb -d wordpress
Unable to find image 'wordpress:latest' locally
latest: Pulling from library/wordpress
8c7716127147: Pull complete
24403a1f6855: Pull complete
e1cf44d6017a: Downloading [==================================>                ]  87.26MB/117.8MB
2489d5e860a7: Download complete
0248257cbd51: Download complete
dd53cf9bf4cf: Download complete
a139c2f3234a: Download complete
6571cfdbe5b2: Download complete
8d83c968ca9a: Download complete
fddb92e888a7: Download complete
749b92ea0995: Download complete
4eed3454c20c: Download complete
00ef78e422f0: Download complete
004f06ab2f6c: Download complete
4f4fb700ef54: Download complete
ad1401f3880c: Download complete
aff9b4e3d718: Downloading [==========================================>        ]  31.06MB/33.93MB
439737a78ff3: Download complete
14ca90809f9f: Download complete
aff9b4e3d718: Pull complete
439737a78ff3: Pull complete
14ca90809f9f: Pull complete
a6a46d84ee78: Pull complete
089dfe755d3d: Pull complete
1ad617015078: Pull complete
fdfd46fc530c: Pull complete
1dc412ee8328: Pull complete
Digest: sha256:d34b44296d24686bf1a9c70c435b1640202e8e97d824aad9997fc63a61ac5cbb
Status: Downloaded newer image for wordpress:latest
755ed53b035819cf2477adfdba3d2ebd36791173b3b91e5375924598fa5d46d7
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on en
dpoint nuestrowp (81ce67d66cada1376c18f220a9e81988447cb6753afd87e75b5f05de79d96d7d): Bind for 0.0.0.0:8080 failed: port is alrea
dy allocated

Run 'docker run --help' for more information
miriam                                  :~$ docker rm -f nuestrowp
nuestrowp
miriam                                  :~$ docker run --name nuestrowp --network redwp -p 8081:80 -e WORDPRESS_DB_HOST=nuestromar
iadb -e WORDPRESS_DB_USER=cefireuser -e WORDPRESS_DB_PASSWORD=cefirepass -e WORDPRESS_DB_NAME=cefiredb -d wordpress
d915fb4b95e199997290b1c54dbf7ab9a23ad06166ef09567bbf025ee81df6ff
miriam                                  :~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS
      NAMES
d915fb4b95e1   wordpress      "docker-entrypoint.s…"   21 seconds ago   Up 20 seconds   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
      nuestrowp
ed45fcd6ca26   mariadb:10.6   "docker-entrypoint.s…"   53 minutes ago   Up 53 minutes   3306/tcp
      nuestromariadb
07bb14b31433   theasp/novnc   "/app/entrypoint.sh"     3 hours ago      Up 3 hours      0.0.0.0:8080->8080/tcp, [::]:8080->8080/
tcp   elated_raman
```

Estaba dando error en el puerto 8080 y se cambió al 8081

4. INSTALACIÓN WORDPRESS

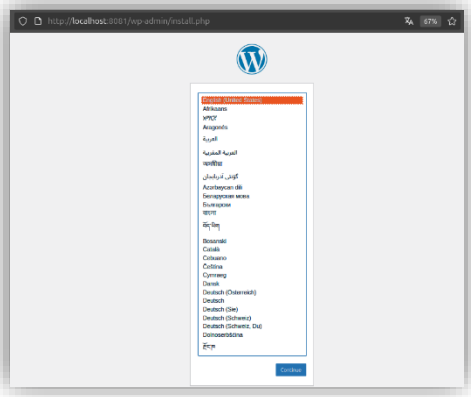


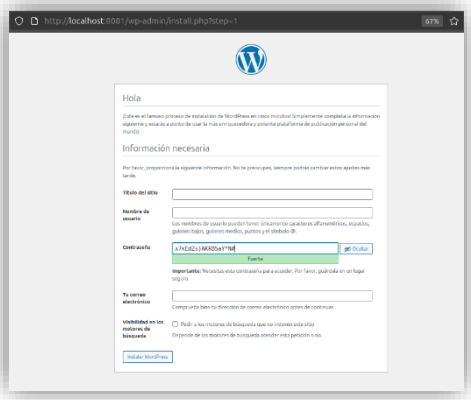


5. MIGRANDO CONTENEDOR MARIADB DE 10.6 A 10.7

```
miriam                          :~$ docker stop nuestromariadb
nuestromariadb
miriam                          :~$ docker rm nuestromariadb
nuestromariadb
```

```
miriam                          :~$ docker run --name nuestromariadb --network redwp -v /home/miriam/mariadbdata:/var/lib/
mysql -e MARIADB_ROOT_PASSWORD=cefireroot -e MARIADB_USER=cefireuser -e MARIADB_PASSWORD=cefirepass -e MARIADB_DATABASE=cefiredb
 -d mariadb:10.7
Unable to find image 'mariadb:10.7' locally
10.7: Pulling from library/mariadb
99803d4b97f3: Pull complete
b8bc823a83fd: Pull complete
16685f710f5d: Pull complete
b5660ff63058: Pull complete
8b13582f0741: Pull complete
7d98acc143e6: Pull complete
6e4fbbea63e0: Pull complete
d3da17671559: Pull complete
Digest: sha256:9a48ac9f196f3d4fd6fea2cab59a49df9e7ca459bf14b2f7b85a0e38a5454571
Status: Downloaded newer image for mariadb:10.7
6213257960d5d057f23d90a191922a4e8b8a0aef1fc34ba8dbb40da7d6c842a8
```





ACTIVIDAD 03

UD 06. Caso práctico 01 - Wordpress + MariaDB

Se va a poner en marcha el popular CMS Wordpress. Para ello se usará un fichero “docker-compose.yml” comentado que nos pondrá en marcha dos contenedores: el primero utilizando “Apache + PHP”, junto con una versión instalada de Wordpress, mientras que el segundo contendrá un servidor de bases de datos MariaDB.

Contenido del fichero “docker-compose.yml”(explicación por línea →)

#Versión del fichero docker-compose 3.9. No obligatorio desde la versión de docker-compose 1.27.0

version: "3.9" # →Define la versión del esquema de docker-compose. Aunque desde la versión 1.27.0 no es obligatorio, incluirlo ayuda a mantener compatibilidad y claridad

#Indicamos los servicios a lanzar

services: # →Aquí comienzan los contenedores que se van a lanzar. Cada uno se define como un servicio independiente.

#Plantill del servicio "db"

db:

#Se basa en la imagen "mariadb", version 10.11.2

image: mariadb:10.11.2 #-->Crea un contenedor basado en la imagen oficial de MariaDB, versión 10.11.2.

#Mapea en el volumen "db_data" el directorio "/var/lib/mysql", lo que da persistencia al contenido de

#Wordpress almacenado en la base de datos

volumes:

- **db_data:/var/lib/mysql** #-->Mapea el volumen db_data al directorio interno donde MariaDB guarda los datos. Esto garantiza persistencia: si el contenedor se borra, los datos siguen ahí.

#Indica que siempre que el servicio finalice, se reiniciará

restart: always #-->Si el contenedor se detiene por error, se reinicia automáticamente. Muy útil en entornos de producción o pruebas continuas.

#Define un conjunto de variables de entorno para estos contenedores,

#indicando password de root de mariadb, nombre de base de datos,

usuario con permisos root (necesario para conexiones remotas) y password de ese usuario

environment: #-->Variables de entorno que configuran la base de datos

MARIADB_ROOT_PASSWORD: somewordpress #-->clave del superusuario

MARIADB_DATABASE: wordpress #-->nombre de la base de datos

MARIADB_USER: wordpress

MARIADB_PASSWORD: wordpress #-->usuario con permisos para acceder desde WordPress.

#Plantilla del servicio "wordpress"

wordpress:

#Indicamos que para lanzar este servicio, debe estar en marcha "db"

depends_on:

- db #-->Indica que este servicio depende de que db esté activo. No garantiza que esté listo, pero sí que se inicia antes.

#Indicamos que basa en la imagen "wordpress", version "latest"

image: wordpress:latest #-->Usa la última versión de WordPress

#Indicamos que el puerto 80 del contenedor se mapea con el puerto 8000 del anfitrion

ports:

- "8000:80" #-->Mapea el puerto 80 del contenedor (donde WordPress sirve contenido) al puerto 8000 del host. Así puedes acceder desde http://localhost:8000.

#Indica que siempre que el servicio finalice, se reiniciará

restart: always #-->se reinicia automáticamente si falla.

#Definimos “variables de entorno”. Definimos donde conectarnos a la base de datos,

#usuario de la base de datos, password de la base de datos y nombre de la base de datos

environment:

WORDPRESS_DB_HOST: db:3306 #-->nombre del servicio db y puerto 3306.

WORDPRESS_DB_USER: wordpress

WORDPRESS_DB_PASSWORD: wordpress

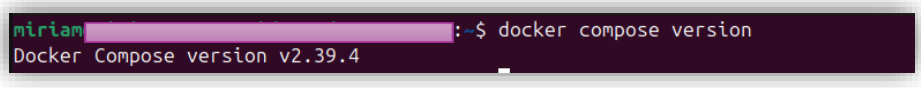
WORDPRESS_DB_NAME: wordpress #-->deben coincidir con los definidos en MariaDB.

#Indicamos los volúmenes creados y compartidos a lo largo del fichero docker-compose.yml

volumes:

db_data: #-->Define el volumen db_data que se usó en el servicio db. Docker lo crea automáticamente si no existe

1. PONER EN MARCHA EL SISTEMA
   - Comprobación Docker Compose instalado



- Creando el fichero “docker-compose.yml”

```
miriam                          :~$ mkdir wordpress-docker
miriam                          :~$ nano docker-compose.yml
```

```
 GNU nano 7.2                                  docker-compose.yml *
version: "3.9"

services:
  db:
    image: mariadb:10.11.2
    volumes:
      - db_data:/var/lib/mysql
    restart: always
    environment:
      MARIADB_ROOT_PASSWORD: somewordpress
      MARIADB_DATABASE: wordpress
      MARIADB_USER: wordpress
      MARIADB_PASSWORD: wordpress

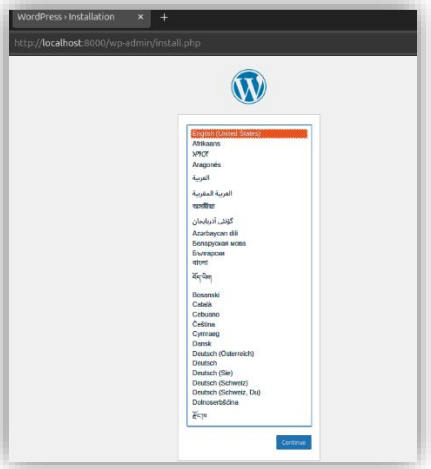
  wordpress:
    depends_on:
      - db
    image: wordpress:latest
    ports:
      - "8000:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress
      WORDPRESS_DB_NAME: wordpress
```

Claves más importantes:

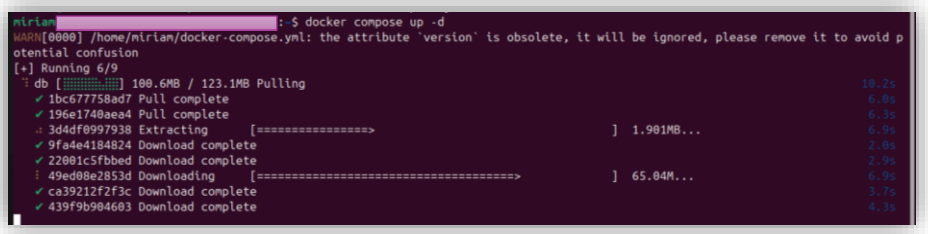
- Indicamos que los servidores de bases de datos (“db”)
  - Enlacen su información a un volumen, dotándolo de persistencia.
  - Definan una serie de variables de entorno definiendo usuarios, contraseñas y bases de datos a usar.
- Indicamos que los servidores con Apache + PHP + Wordpress (“wordpress”):
  - Para iniciarse, debe iniciarse antes un servicio “db”.
  - Establece variables de entorno definiendo valores para la conexión de base de datos de Wordpress.
  - Enlaza puerto 80 del contenedor a puerto 8000 del anfitrión.

```
miriam                          :~$ docker compose up -d
WARN[0000] /home/miriam/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid p
otential confusion
[+] Running 9/9
 ✔ db Pulled                                                                                                              19.2s
   ✔ 1bc677758ad7 Pull complete                                                                                            6.0s
   ✔ 196e1740aea4 Pull complete                                                                                            6.3s
   ✔ 3d4df0997938 Pull complete                                                                                            8.4s
   ✔ 9fa4e4184824 Pull complete                                                                                            8.8s
   ✔ 22001c5fbbed Pull complete                                                                                            9.1s
   ✔ 49ed08e2853d Pull complete                                                                                           15.1s
   ✔ ca39212f2f3c Pull complete                                                                                           15.3s
   ✔ 439f9b904603 Pull complete                                                                                           15.5s
[+] Running 4/4
 ✔ Network miriam_default        Created                                                                                   0.4s
 ✔ Volume miriam_db_data         Created                                                                                   0.2s
 ✔ Container miriam-db-1         Started                                                                                   7.4s
 ✔ Container miriam-wordpress-1  Started                                                                                   4.3s
```

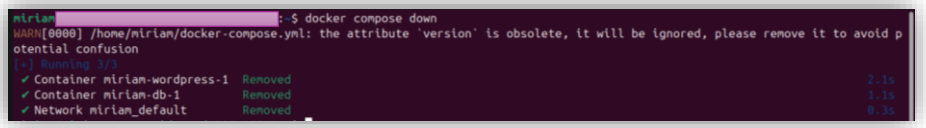
- Probar que todo es correcto accediendo a http://localhost:8000

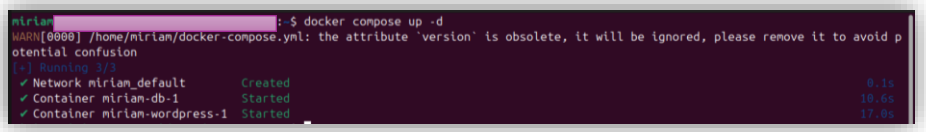


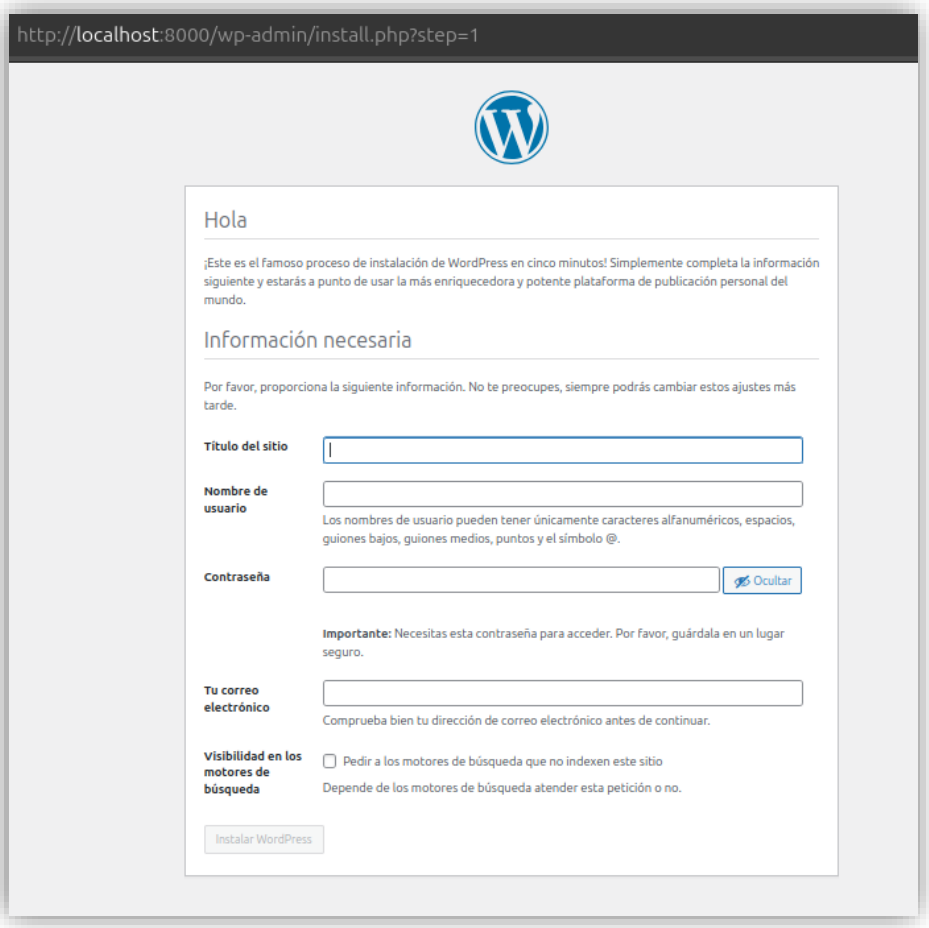
2. PARANDO EL SISTEMA



3. RE-LANZANDO EL SISTEMA







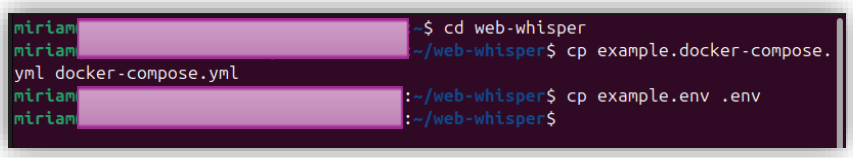
UD 06. Caso práctico 04 - Whisper AI y Web Whisper

Whisper AI es un software que permite transcribir audio a Texto. Este software fue creado por OpenAI (empresa creadora de ChatGPT) y liberado junto con varios modelos de procesamiento de lenguaje muy precisos. Actualmente, es una de las mejores soluciones “self-hosted” para convertir audio a texto.

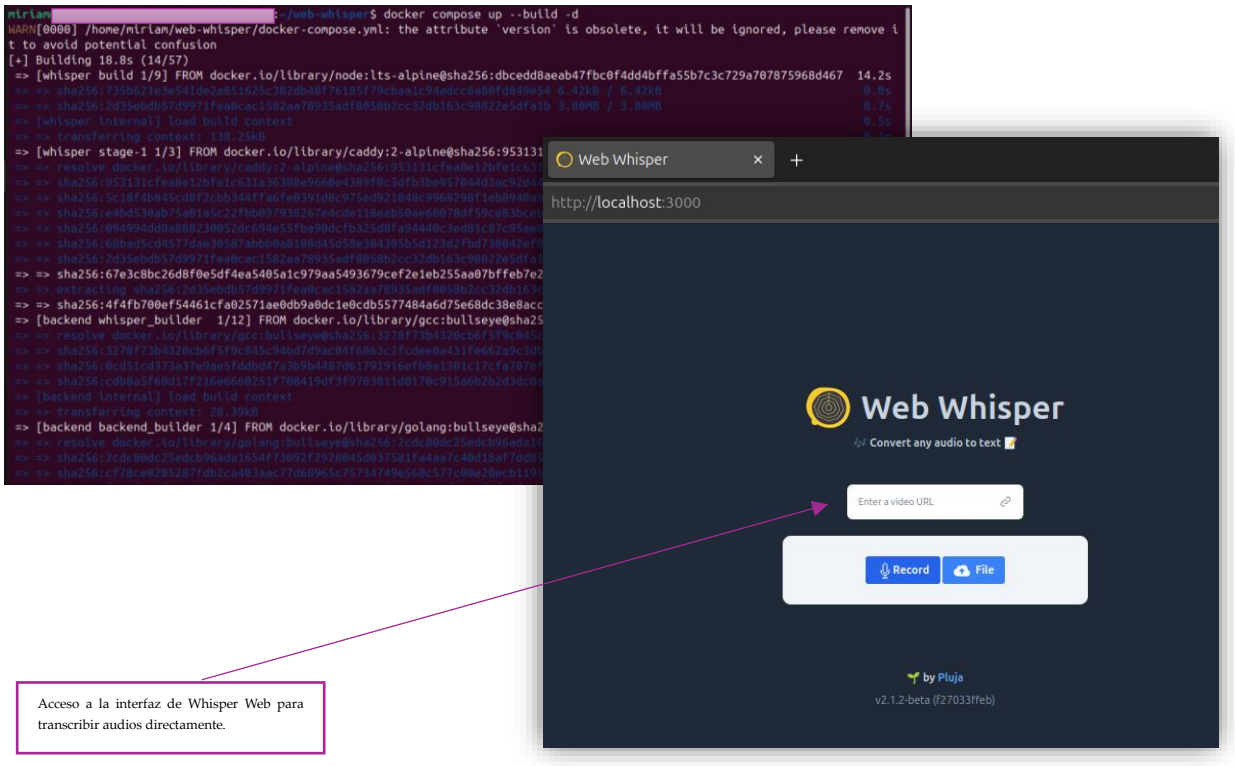
Whisper Web es una web, que proporciona una interfaz gráfica web para mandar audios a transcribir a Whisper AI. Su repositorio es https://codeberg.org/pluja/web-whisper

En este caso práctico vamos a poner en marcha una versión Dockerizada extraido de su repositorio siguiendo las instrucciones de https://codeberg.org/pluja/web-whisper/wiki/Self-Hosting

1. Pasos pevios:
   git clone https://codeberg.org/pluja/web-whisp



2. WHISPER AI Y WHISPER WEB



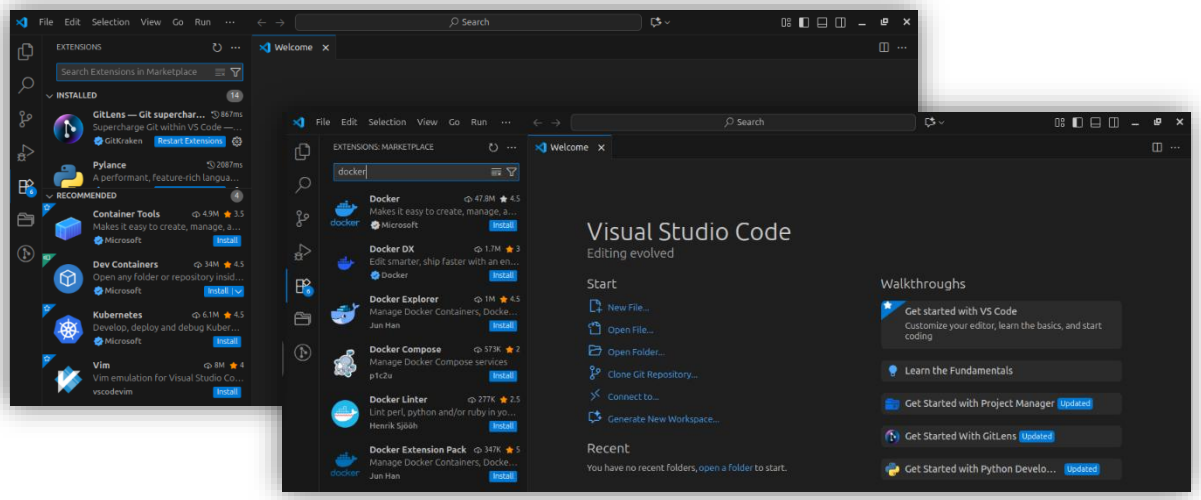
ACTIVIDAD 04

UD 07. Caso práctico 01 - Desarrollando con Visual Studio Code en un contenedor

Se va a utilizar el editor multiplataforma “Visual Studio Code” junto con plugins relacionados con Docker para poder desarrollar una sencilla aplicación web en un contenedor.

1. Instalar herramientas necesarias
   - Visual Studio Code
     https://code.visualstudio.com/

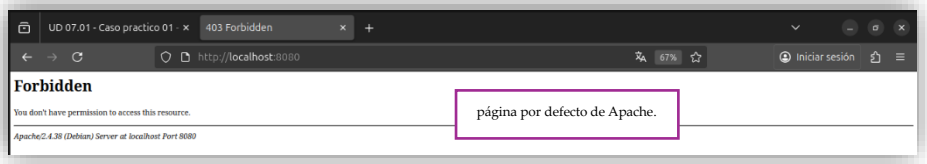
   - Aprende a usarlo en code.visualstudio.com/learn
     https://code.visualstudio.com/docs/getstarted/getting-started



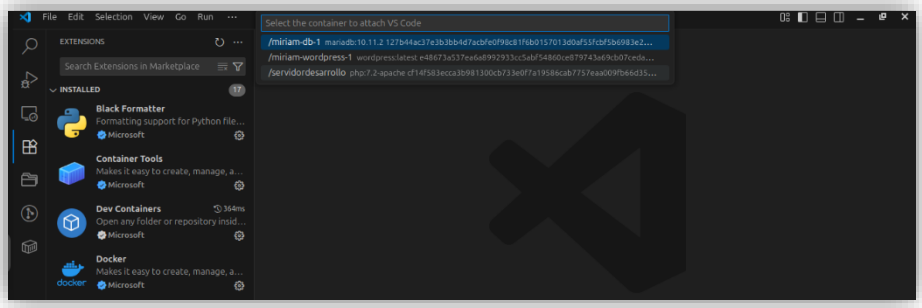
2. CREAR Y EDITAR APLICACIÓN EN PHP DENTRO DE UN CONTENEDOR



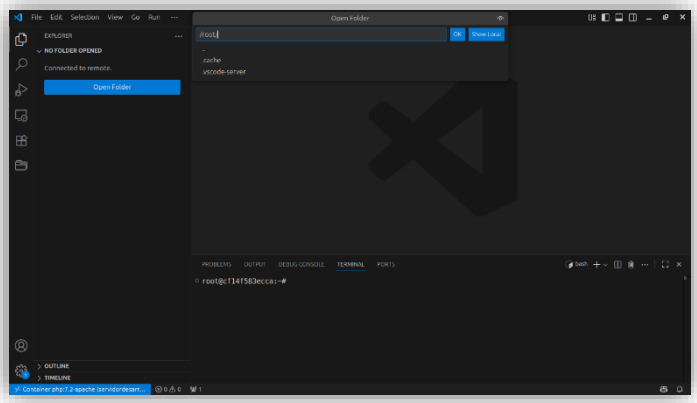
Una vez creado nuestro contenedor probaremos a acceder a http://localhost:8080

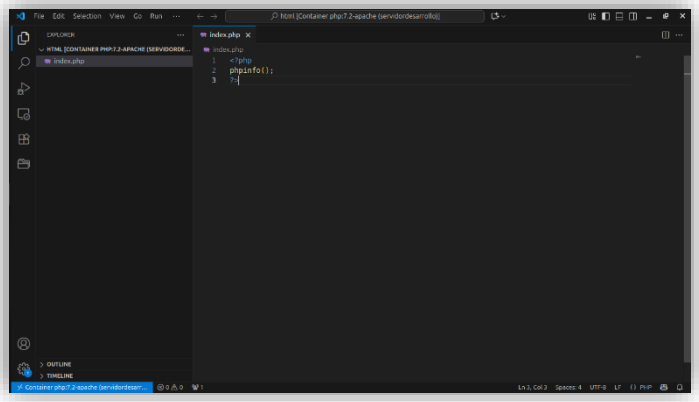


3. Conectar VS Code al contenedor
   - Abrir VS Code
   - Clic en el icono de la ballena(Docker)
   - ir a la pestaña Containers
   - Clic derecho sobre servidordesarrollo y selecciona Attach Visual Studio Code o Ctrl+Shift+P →Escribir: *Dev Containers: Attach to Running Container...*
     Y seleccionar el contenedor servidordesarrollo



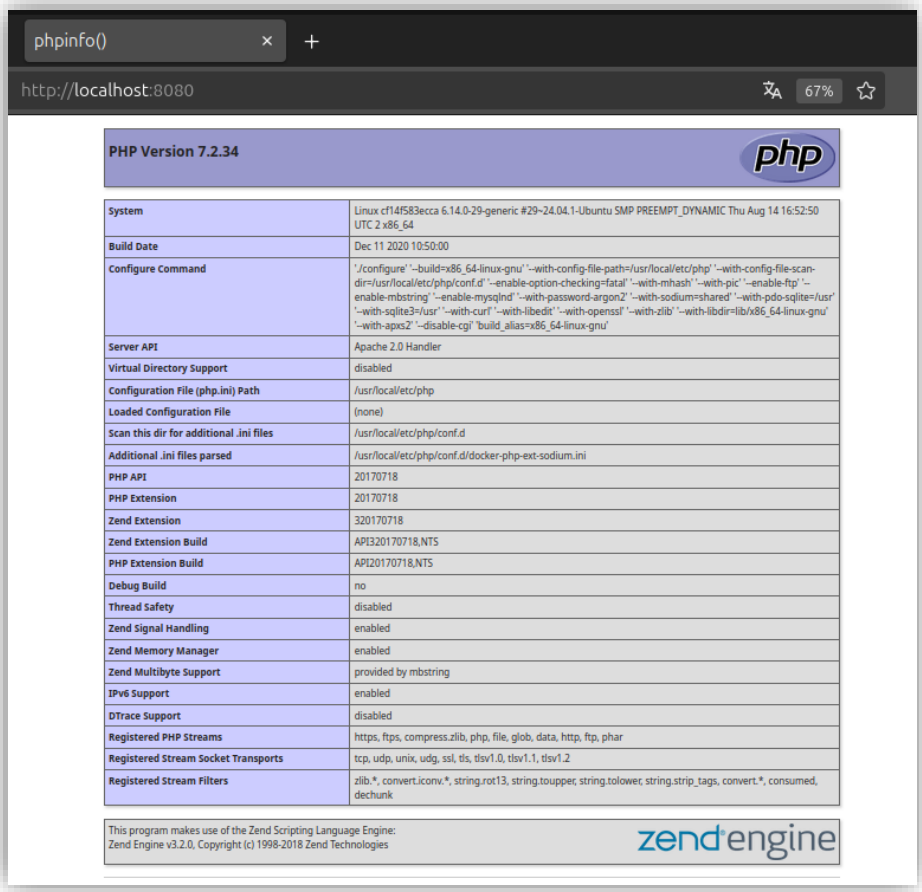
4. Crear archivo PHP dentro del contenedor
   - Navega al directorio /var/www/html
   - Crea un archivo llamado index.php





5. Verificar en navegador

   http://localhost:8080 → ver la salida de phpinfo() mostrando la configuración del servidor PHP.



phpinfo()

http://localhost:8080

67%

PHP Version 7.2.34

System	Linux cf14f583ecca 6.14.0-29-generic #29~24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Thu Aug 14 16:52:50 UTC 2 x86_64
Build Date	Dec 11 2020 10:50:00
Configure Command	'./configure' '--build=x86_64-linux-gnu' '--with-config-file-path=/usr/local/etc/php' '--with-config-file-scan-dir=/usr/local/etc/php/conf.d' '--enable-option-checking=fatal' '--with-mhash' '--with-pic' '--enable-ftp' '--enable-mbstring' '--enable-mysqlnd' '--with-password-argon2' '--with-sodium=shared' '--with-pdo-sqlite=/usr' '--with-sqlite3=/usr' '--with-curl' '--with-libedit' '--with-openssl' '--with-zlib' '--with-libdir=lib/x86_64-linux-gnu' '--with-apxs2' '--disable-cgi' 'build_alias=x86_64-linux-gnu'
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/usr/local/etc/php
Loaded Configuration File	(none)
Scan this dir for additional .ini files	/usr/local/etc/php/conf.d
Additional .ini files parsed	/usr/local/etc/php/conf.d/docker-php-ext-sodium.ini
PHP API	20170718
PHP Extension	20170718
Zend Extension	320170718
Zend Extension Build	API320170718,NTS
PHP Extension Build	API20170718,NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
IPv6 Support	enabled
DTrace Support	disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2
Registered Stream Filters	zlib.*, convert.iconv.*, string.rot13, string.toupper, string.tolower, string.strip_tags, convert.*, consumed, dechunk

This program makes use of the Zend Scripting Language Engine:
Zend Engine v3.2.0, Copyright (c) 1998-2018 Zend Technologies

zend engine

ACTIVIDAD 05

Lea el cómic de "Kubernetes" que está disponible en la siguiente url: https://cloud.google.com/kubernetes-engine/kubernetescomic?hl=es-419 (o en el PDF Adjunto si falla el enlace). Resumir en un máximo de 8 líneas que es Kubernetes y porqué razones puede ser útil.



Kubernetes es una plataforma de código abierto para automatizar el despliegue, escalado y gestión de aplicaciones en contenedores. Organiza las implementaciones con una API de código abierto, agrupando los contenedores en Pods, distribuidos en *nodos* dentro de un *cluster*. Permite escalar servicios fácilmente, reiniciar contenedores fallidos, realizar actualizaciones sin interrupciones y mantener alta disponibilidad. Es útil porque mejora la eficiencia operativa, facilita el desarrollo continuo, y permite gestionar infraestructuras complejas. Ideal para entornos cloud, microservicios y aplicaciones que requieren flexibilidad y robustez.

BIBLIOGRAFIA

- Apunts del Professor.
- https://sergarb1.github.io/CursoIntroduccionADocker
- https://cloud.google.com/kubernetes-engine/kubernetes-comic